CMPE 125 Laboratory Report

Lab 5: 4-bit CLA Adder Test Bench

Name: Yixu Liu **Student ID:** 017406857
Instructor: Jahan Ghofraniha **Date:** October 14, 2026

---

1 Logic Circuit Design and Calculations

1.1 Objective and circuit specification

The objective of this lab is to verify the structural 4-bit carry-lookahead adder (CLA) from Lab 4 using two Verilog testbenches: a Type 2 self-checking testbench and a Type 3 testbench that reads expected results from a separate file [1, 2]. Both testbenches apply the same four additions. The recorded laboratory simulations use Vivado Simulator (XSim), with each testbench selected separately as the simulation top.

The combinational circuit accepts unsigned inputs $A[3:0]$, $B[3:0]$, and carry-in C_{in} (`Ci`). It produces the four-bit sum $S[3:0]$, carry-out C_{out} (`Co`), group propagate PG, and group generate GG. Correct addition requires

$$\{C_{\text{out}}, S[3:0]\} = A + B + C_{\text{in}}. \tag{1}$$

The result is five bits wide because 15 + 15 + 1 = 31. The carry-out represents unsigned carry beyond bit 3; it is not a signed-overflow flag.

1.2 Bit propagate, generate, and sum

For bit position $i \in \{0, 1, 2, 3\}$, define

$$P_i = A_i \oplus B_i, \qquad G_i = A_i B_i, \tag{2}$$

$$S_i = P_i \oplus C_i, \qquad C_{i+1} = G_i + P_i C_i, \tag{3}$$

where $C_0 = C_{\text{in}}$. In the Boolean equations, + means OR, juxtaposition means AND, and $\oplus$ means XOR. A generate signal is high when both operand bits are 1. With the XOR definition used by this design, a propagate signal is high when exactly one operand bit is 1.

1.3 Carry-lookahead equations

Substituting each carry expression into the next removes the need to wait for a chain of intermediate carry outputs:

$$C_1 = G_0 + P_0 C_0, \tag{4}$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0, \tag{5}$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0, \tag{6}$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0. \tag{7}$$

The group signals factor the final carry into

$$PG = P_3 P_2 P_1 P_0, \tag{8}$$

$$GG = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0, \tag{9}$$

$$C_{\text{out}} = C_4 = GG + PG C_{\text{in}}. \tag{10}$$

Thus, PG indicates propagation through the entire four-bit group, while GG indicates that the group generates a carry even when $C_{\text{in}} = 0$. The implementation uses gate primitives for these equations. The behavioral simulations verify logic values; they do not measure a physical speedup.

1.4 Structural implementation

The top module, `CLA4`, instantiates four `GPFullAdder` modules and one `CLALogic` module. Each full-adder slice uses one AND gate and two XOR gates to produce G_i, P_i, and S_i. The lookahead module uses AND/OR gate networks to compute all carries and the group outputs. The Verilog wire `C[0]` corresponds to mathematical C_1, `C[1]` to C_2, and `C[2]` to C_3.

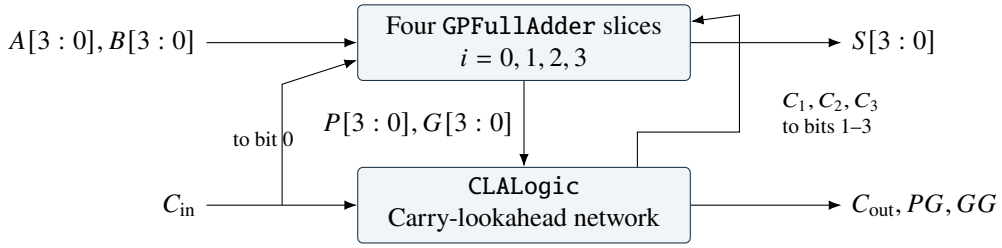


Figure 1: Module-level organization of the structural CLA. The carry inputs affect the sum outputs; P_i and G_i depend only on the operands.

1.5 Expected results for the four additions

Table 1 lists the expected values before simulation. All bit vectors are written most-significant bit first. The carry column explicitly lists C_1, C_2, C_3, C_4 in that order.

Case	Addition	$P[3:0]$	$G[3:0]$	$C_1C_2C_3C_4$	$\{C_o, S\}$	PG	GG
1	0 + 0 + 0 = 0	`0000`	`0000`	`0000`	`00000`	0	0
2	3 + 5 + 0 = 8	`0110`	`0001`	`1110`	`01000`	0	0
3	9 + 6 + 1 = 16	`1111`	`0000`	`1111`	`10000`	1	0
4	15 + 15 + 0 = 30	`0000`	`1111`	`1111`	`11110`	0	1

Table 1: Calculated arithmetic, carry, and group-signal results.

Case 1: zero addition. For $A = B =$ `0000` and $C_0 = 0$, all P_i, G_i, and carries are 0. Therefore $S =$ `0000`, $C_o = 0$, $PG = 0$, and $GG = 0$.

Case 2: internal carry propagation. For $A =$ `0011` and $B =$ `0101`, $P =$ `0110` and $G =$ `0001`. Bit 0 generates a carry, and bits 1 and 2 propagate it: $C_1 = C_2 = C_3 = 1$. Since $P_3 = G_3 = 0$, $C_4 = 0$. The carry-input vector for the four sum bits is $(C_3C_2C_1C_0) =$ `1110`, giving $S =$ `0110` $\oplus$ `1110` = `1000`.

Case 3: carry-in propagates through the group. For $A =$ `1001` and $B =$ `0110`, $P =$ `1111` and $G =$ `0000`. With $C_0 = 1$, every carry is 1 and $S =$ `1111` $\oplus$ `1111` = `0000`. Thus $PG = 1$, $GG = 0$, and $C_o = GG + PGC_0 = 1$, producing `10000`.

Case 4: group carry generation. For $A = B =$ `1111`, $P =$ `0000` and $G =$ `1111`. Each bit generates a carry, so $C_1 = C_2 = C_3 = C_4 = 1$ even though $C_0 = 0$. Therefore $S =$ `0000` $\oplus$ `1110` = `1110`, $PG = 0$, and $GG = 1$, producing `11110`.

2 Testbench Design

2.1 Type 2: self-checking testbench

The module `self_check_tb` instantiates `CLA4` and applies the four cases sequentially in one `initial` block. Each input combination is held for 10 ns before the concatenated output `{Co, S}` is checked against a five-bit expected value. The case-inequality operator `!==` detects a mismatch even if an output contains `x` or `z`. The carry-in test compares the output against `5'd16`.

The four comparisons occur at 10, 20, 30, and 40 ns; `$finish` terminates the simulation at 40 ns. This testbench automatically checks the sum and carry-out. Its PG and GG signals are visible in the waveform but are not included in its `if` conditions.

2.2 Type 3: testbench using an external vector file

The module `testVector_tb`, stored in `testVec_tb.v`, loads four 16-bit words with `$readmemb`. Each word contains

$$\underbrace{A}_{4}\ \underbrace{B}_{4}\ \underbrace{C_i}_{1}\ \underbrace{\{C_o, S\}_{\text{expected}}}_{5}\ \underbrace{PG_{\text{expected}}}_{1}\ \underbrace{GG_{\text{expected}}}_{1}, \qquad 4+4+1+5+1+1 = 16 \text{ bits.}$$

The submitted `test_vector.tv` contains:

Listing 1: The four external test vectors.

```
0000_0000_0_00000_0_0
0011_0101_0_01000_0_0
1001_0110_1_10000_1_0
1111_1111_0_11110_0_1
```

The array is declared `reg [15:0] testvectors [0:3]`. Underscores separate fields for readability. The source uses the full vector-file path from the laboratory computer. That path must point to the actual file when the project is moved to another computer.

A 10 ns testbench clock schedules the operations. Reset remains high until 27 ns to prevent premature checking. After reset is released, each vector is applied 1 ns after a rising clock edge and checked on the following falling edge. This provides 4 ns between stimulus and comparison. The adder itself has no clock or reset; these signals control only the testbench.

Case	Apply time	Check time	Expected $\{C_o, S\}$	PG	GG
1	31 ns	35 ns	`00000`	0	0
2	41 ns	45 ns	`01000`	0	0
3	51 ns	55 ns	`10000`	1	0
4	61 ns	65 ns	`11110`	0	1

Table 2: Stimulus and checking schedule derived from the Type 3 source.

The Type 3 comparison uses `!==` to check all seven output bits: `{Co, S, PG, GG}` against `{expected, PGexpected, GGexpected}`. After four comparisons, it prints the test and error counts and stops at 65 ns. Both testbenches use delays and system tasks for simulation and are not intended for synthesis [2].

3 Compilation and Simulation Console Evidence

The supplied Vivado screenshots document successful simulation of both testbench tops. Figure 3 also shows completion of static elaboration. It contains a `LIBRARY_PATH` environment warning; no HDL error is shown, and the simulation completes with zero test errors. The displayed `run 1000ns` is the requested run interval: the actual runs end earlier when each testbench executes `$finish`.

```
# run 1000ns
Test 1 passed.
Test 2 passed.
Test 3 passed.
Test 4 passed.
$finish called at time : 40 ns : File "/home/017406857@SJSUAD/lab5/lab5.srcs/sim_1/new/self_check_tb.v" Line 67
INFO: [USF-XSim-96] XSim completed. Design snapshot 'self_check_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:05 . Memory (MB): peak = 10479.852 ; gain = 53.184 ; free physical = 22454 ; free virtual = 2908
```

Figure 2: Excerpt from the original Vivado/XSim console for `self_check_tb`: Tests 1–4 pass, and `$finish` is called at 40 ns.

```
INFO: [USF-XSim-3] XSim::Elaborate design
INFO: [USF-XSim-61] Executing 'ELABORATE' step in '/home/017406857@SJSUAD/lab5/lab5.sim/sim_1/behav/xsim'
xelab --incr --debug typical --relax --mt 8 -L xil_defaultlib -L unisims_ver -L unimacro_ver -L secureip --snapshot testVector_tb_behav xil_defaultlib.t
Vivado Simulator v2025.2
Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
Copyright 2022-2025 Advanced Micro Devices, Inc. All Rights Reserved.
Running: /opt/Xilinx/2025.2/Vivado/bin/unwrapped/lnx64.o/xelab --incr --debug typical --relax --mt 8 -L xil_defaultlib -L unisims_ver -L unimacro_ver -L
Using 8 slave threads.
WARNING: [XSIM 43-3431] One or more environment variables have been detected which affect the operation of the C compiler. These are typically not set i
    LIBRARY_PATH
Starting static elaboration
Pass Through NonSizing Optimizer
Completed static elaboration
INFO: [XSIM 43-4323] No Change in HDL. Linking previously generated obj files to create kernel
INFO: [USF-XSim-69] 'elaborate' step finished in '1' seconds
INFO: [USF-XSim-4] XSim::Simulate design
INFO: [USF-XSim-61] Executing 'SIMULATE' step in '/home/017406857@SJSUAD/lab5/lab5.sim/sim_1/behav/xsim'
INFO: [USF-XSim-98] *** Running xsim
   with args "testVector_tb_behav -key {Behavioral:sim_1:Functional:testVector_tb} -tclbatch {testVector_tb.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Time resolution is 1 ps
source testVector_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to '
#   }
# }
# run 1000ns
Test 1 PASSED: Sum=00000 PG=0 GG=0
Test 2 PASSED: Sum=01000 PG=0 GG=0
Test 3 PASSED: Sum=10000 PG=1 GG=0
Test 4 PASSED: Sum=11110 PG=0 GG=1
4 tests completed with 0 errors
$finish called at time : 65 ns : File "/home/017406857@SJSUAD/lab5/lab5.srcs/sim_1/new/testVec_tb.v" Line 87
INFO: [USF-XSim-96] XSim completed. Design snapshot 'testVector_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

Figure 3: Excerpt from the original Vivado/XSim elaboration and Type 3 simulation output: all four expected sum, *PG*, and *GG* combinations pass; the final message reports “4 tests completed with 0 errors” at 65 ns.

4 Output Simulation Waveforms

4.1 Type 2 waveform

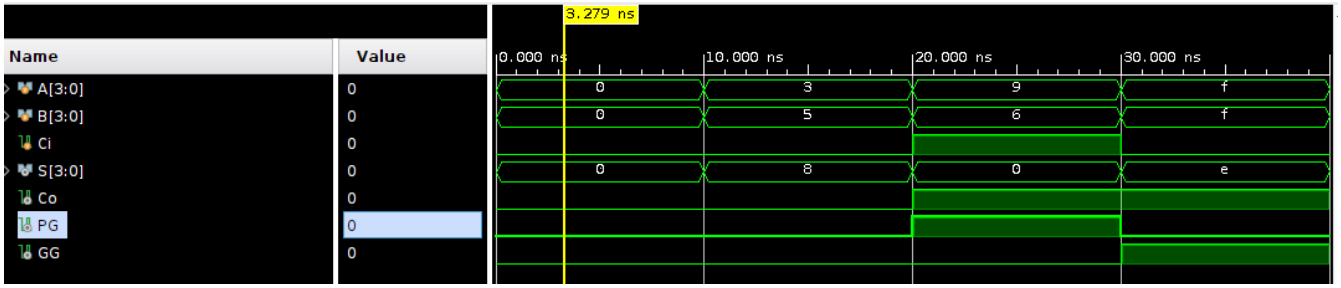


Figure 4: Original self-checking testbench waveform, showing the four cases over 0–40 ns. Bus values use hexadecimal notation; unused lower space is cropped.

The intervals 0–10, 10–20, 20–30, and 30–40 ns correspond to Cases 1–4. The sum is respectively `0`, `8`, `0`, and `e` (14 decimal), while C_o is 0, 0, 1, and 1. Thus the last two full results are 16 + 0 = 16 and 16 + 14 = 30. The PG signal is high only for Case 3, and GG is high only for Case 4, matching the calculations. The left-hand value column reflects the cursor position, not all four tests.

4.2 Type 3 waveform

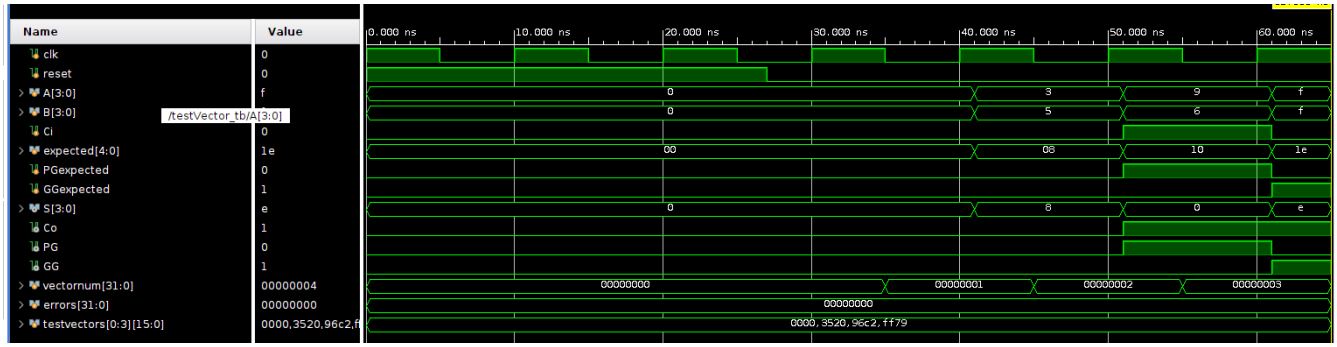


Figure 5: Original vector-based testbench waveform. The expected result, expected group flags, actual outputs, vector count, and error count are displayed together; unused lower space is cropped.

Reset deasserts at 27 ns, and the first vector is applied at 31 ns. Because Case 1 equals the initialized zero inputs, it causes no visible change in A, B, or S. The remaining vectors appear at 41, 51, and 61 ns. The expected bus shows `00`, `08`, `10`, and `1e` in hexadecimal. Its upper bit matches C_o and its lower four bits match S. The actual PG and GG signals follow their expected values. The error counter remains zero, and the final vector count is four.

5 Verification Summary

The structural 4-bit CLA produced the correct five-bit result for all four required additions: 0 + 0 = 0, 3 + 5 = 8, 9 + 6 + 1 = 16, and 15 + 15 = 30. The Type 2 testbench reported four passing arithmetic checks, while the Type 3 testbench verified the arithmetic result and both group signals, reporting four tests with zero errors. The waveforms agree with the calculated carries: Case 3 propagates the incoming carry through all four bits ($PG = 1$, $GG = 0$), and Case 4 generates a carry within the group ($PG = 0$, $GG = 1$). Together, the calculations, console messages, and waveforms verify the implemented logic for the selected cases. These four cases cover several important carry behaviors but do not exhaust all $16 \times 16 \times 2 = 512$ input combinations or establish physical propagation delay.

6 Separate Verilog Source Files

Appendix A reproduces the complete module bodies from the supplied code [3]. The report is also accompanied by the original source files in the `verilog/` directory. Each testbench should be run separately with the same three design files. The `test_vector.tv` file must also be available to the Type 3 simulation.

File	Purpose
`GPFullAdder.v`	One-bit propagate/generate full-adder slice.
`CLALogic.v`	Internal carries, carry-out, group propagate, and group generate.
`CLA4.v`	Structural top module connecting four slices to the lookahead logic.
`self_check_tb.v`	Type 2 testbench with four explicit arithmetic checks.
`testVec_tb.v`	Type 3 testbench; its module name is `testVector_tb`.
`test_vector.tv`	Four input and expected-output vectors.

Table 3: Source files submitted separately from the report.

6.1 Reproduction notes

For the Type 2 Vivado simulation, add `GPFullAdder.v`, `CLALogic.v`, and `CLA4.v` as design sources, add `self_check_tb.v` as a simulation source, and choose `self_check_tb` as the simulation top. Run behavioral simulation until the testbench finishes at 40 ns.

For the Type 3 simulation, use the same design sources with `testVec_tb.v` as the simulation source and `testVector_tb` as the simulation top. In `$readmemb`, set the full path to the local `test_vector.tv` file. Run until the testbench finishes at 65 ns. The submitted source retains the original laboratory path to preserve its correspondence with the supplied evidence.

References

[1] CMPE 125, *Lab 5: 4-bit CLA Adder Test Bench*, supplied laboratory manual, pp. 1–2. File: `Lab 5 Manual.pdf`.

[2] CMPE 125, *Writing a Test Bench for Your Verilog Code*, supplied quick-reference handout, pp. 1–4. File: `TestBench_Reference_Handout.pdf`.

[3] Laboratory project materials supplied with this report: the five Verilog source files, `test_vector.tv`, and the four original Vivado/XSim console and waveform screenshots.

A Verilog Source Code

The original module bodies are reproduced below; tool-generated headers and blank lines are omitted for readability. All sources use `‘timescale 1ns / 1ps`. The separate `.v` files retain their original contents and formatting.

A.1 `GPFullAdder.v`: one-bit full-adder slice

```
module GPFullAdder (
    input  Ai,
    input  Bi,
    input  Cin,
    output G,
    output P,
    output S
);
    xor (P, Ai, Bi);
    and (G, Ai, Bi);
    xor (S, P, Cin);
endmodule
```

A.2 `CLALogic.v`: carry-lookahead logic

```
module CLALogic (
    input  [3:0] G,
    input  [3:0] P,
    input        Ci,
    output [2:0] C,
    output       Co,
    output       PG,
    output       GG
);
    // C1 = G0 + P0*Ci
    wire c1_t1;
    and (c1_t1, P[0], Ci);
    or  (C[0], G[0], c1_t1);
    // C2 = G1 + P1*G0 + P1*P0*Ci
    wire c2_t1;
    wire c2_t2;
    and (c2_t1, P[1], G[0]);
    and (c2_t2, P[1], P[0], Ci);
    or  (C[1], G[1], c2_t1, c2_t2);
    // C3 = G2 + P2*G1 + P2*P1*G0 + P2*P1*P0*Ci
    wire c3_t1;
    wire c3_t2;
    wire c3_t3;
    and (c3_t1, P[2], G[1]);
    and (c3_t2, P[2], P[1], G[0]);
    and (c3_t3, P[2], P[1], P[0], Ci);
    or  (C[2], G[2], c3_t1, c3_t2, c3_t3);
    // PG = P3*P2*P1*P0
    and (PG, P[3], P[2], P[1], P[0]);
    // GG = G3 + P3*G2 + P3*P2*G1 + P3*P2*P1*G0
    wire gg_t1;
    wire gg_t2;
    wire gg_t3;
    and (gg_t1, P[3], G[2]);
    and (gg_t2, P[3], P[2], G[1]);
    and (gg_t3, P[3], P[2], P[1], G[0]);
    or  (GG, G[3], gg_t1, gg_t2, gg_t3);
    // Co = GG + PG*Ci
    wire co_t1;
    and (co_t1, PG, Ci);
    or  (Co, GG, co_t1);
endmodule
```

A.3 CLA4.v: structural top module

```
module CLA4 (
    input  [3:0] A,
    input  [3:0] B,
    input        Ci,
    output [3:0] S,
    output       Co,
    output       PG,
    output       GG
);
    wire [3:0] P;
    wire [3:0] G;
    wire [2:0] C;
    // Four 1-bit GP Full Adders
    GPFullAdder FA0 (
        A[0],
        B[0],
        Ci,
        G[0],
        P[0],
        S[0]
    );
    GPFullAdder FA1 (
        A[1],
        B[1],
        C[0],
        G[1],
        P[1],
        S[1]
    );
    GPFullAdder FA2 (
        A[2],
        B[2],
        C[1],
        G[2],
        P[2],
        S[2]
    );
    GPFullAdder FA3 (
        A[3],
        B[3],
        C[2],
        G[3],
        P[3],
        S[3]
    );
    // Carry Lookahead Logic
    CLALogic CLA (
        G,
        P,
        Ci,
        C,
        Co,
        PG,
        GG
    );
endmodule
```

A.4 `self_check_tb.v`: Type 2 testbench

```
module self_check_tb();
    reg [3:0] A, B;
    reg Ci;
    wire [3:0] S;
    wire Co, PG, GG;
    // Instantiate Device Under Test
    CLA4 dut(A, B, Ci, S, Co, PG, GG);
    // Apply inputs and check results
    initial begin
        // Test 1: 0 + 0 = 0
        A = 4'd0; B = 4'd0; Ci = 0; #10;
        if ({Co, S} !== 5'd0)
            $display("Test 1 failed.");
        else
            $display("Test 1 passed.");
        // Test 2: 3 + 5 = 8
        A = 4'd3; B = 4'd5; Ci = 0; #10;
        if ({Co, S} !== 5'd8)
            $display("Test 2 failed.");
        else
            $display("Test 2 passed.");
        // Test 3: 9 + 6 + 1 = 16
        A = 4'd9; B = 4'd6; Ci = 1; #10;
        if ({Co, S} !== 5'd16)
            $display("Test 3 failed.");
        else
            $display("Test 3 passed.");
        // Test 4: 15 + 15 = 30
        A = 4'd15; B = 4'd15; Ci = 0; #10;
        if ({Co, S} !== 5'd30)
            $display("Test 4 failed.");
        else
            $display("Test 4 passed.");
        $finish;
    end
endmodule
```

A.5 `testVec_tb.v`: Type 3 testbench

```
module testVector_tb();
    reg clk, reset;
    reg [3:0] A, B;
    reg Ci;
    reg [4:0] expected;
    reg PGexpected, GGexpected;
    wire [3:0] S;
    wire Co, PG, GG;
    integer vectornum, errors;
    // 16-bit test vectors
    reg [15:0] testvectors [0:3];
    // Instantiate Device Under Test
    CLA4 dut(A, B, Ci, S, Co, PG, GG);
    // Generate clock
    always begin
        clk = 1; #5;
        clk = 0; #5;
    end
    // Load test vectors and initialize
    initial begin
        $readmemb(
            "/home/017406857@SJSUAD/Downloads/test_vector.tv",
            testvectors
        );
        vectornum = 0;
        errors = 0;
        reset = 1;
        A = 0;
        B = 0;
        Ci = 0;
        expected = 0;
        PGexpected = 0;
        GGexpected = 0;
        #27;
        reset = 0;
    end
    // Apply inputs on rising edge
    always @(posedge clk) begin
        if (!reset && vectornum < 4) begin
            #1;
            {A, B, Ci, expected, PGexpected, GGexpected}
                = testvectors[vectornum];
        end
    end
    // Check outputs on falling edge
    always @(negedge clk) begin
        if (!reset && vectornum < 4) begin
            if ({Co, S, PG, GG} !==
                {expected, PGexpected, GGexpected}) begin
                $display("Test %0d FAILED", vectornum + 1);
                $display("A=%b B=%b Ci=%b", A, B, Ci);
                $display("Actual:   Sum=%b PG=%b GG=%b",
                         {Co, S}, PG, GG);
                $display("Expected: Sum=%b PG=%b GG=%b",
                         expected, PGexpected, GGexpected);
                errors = errors + 1;
            end
            else begin
                $display("Test %0d PASSED: Sum=%b PG=%b GG=%b",
                         vectornum + 1, {Co, S}, PG, GG);
            end
            vectornum = vectornum + 1;
            if (vectornum == 4) begin
                $display("%0d tests completed with %0d errors",
                         vectornum, errors);
                $finish;
            end
        end
    end
endmodule
```